

IFRS 17 Term Life Valuation Engine

Actuarial Methodology and Validation Report

Prepared by: Şevval Zerda Höçük
Project type: Actuarial Science CV / Portfolio Project
Date: 15.05.2026

Python 3.11+ • pandas • NumPy • Pydantic • pytest • openpyxl

Table of Contents

- 1. Executive Summary
- 2. Project Objective and Scope
- 3. Product and Portfolio Description
- 4. Data, Assumptions, and Synthetic Data Rationale
- 5. System Architecture
- 6. Projection Methodology
- 7. Cashflow Methodology
- 8. Discounting Methodology
- 9. Best Estimate Liability Methodology
- 10. Risk Adjustment Methodology
- 11. Contractual Service Margin Methodology
- 12. IFRS 17 Grouping and Onerous Contract Testing
- 13. Scenario Analysis
- 14. Audit, Governance, and Reproducibility Features
- 15. Validation and Testing Framework
- 16. Key Strengths of the Project
- 17. Limitations and Future Improvements
- 18. Conclusion
- Appendices

1. Executive Summary

The IFRS 17 Term Life Valuation Engine is a modular Python-based actuarial valuation project designed to demonstrate simplified IFRS 17 measurement concepts for term-life insurance contracts.

The project includes policy-level projection, mortality and lapse modelling, cashflow generation, discounting, Best Estimate Liability, Risk Adjustment, Contractual Service Margin calculation and roll-forward, onerous contract testing, IFRS 17-style grouping, scenario analysis, audit snapshots, output generation, and automated validation testing.

The project is intended as an academic and CV portfolio project for a 4th-year Actuarial Science student. It is not a production IFRS 17 reporting system. Production use would require formal methodology approval, assumption governance, experience-based calibration, independent model validation, data lineage, audit workflow, financial reporting controls, and company-specific IFRS 17 accounting policy.

The use of synthetic data is appropriate for this type of actuarial science project because real policy-level insurance datasets are generally confidential and not publicly available.

2. Project Objective and Scope

The objective of the project is to build a clear, reproducible, and modular actuarial valuation engine for term-life insurance under a simplified IFRS 17-style measurement framework.

The model demonstrates practical actuarial modelling ability rather than official financial reporting. Its scope includes projection, cashflows, discounting, BEL, simplified RA, simplified CSM, grouping, scenarios, audit outputs, and automated tests.

The model should be interpreted as an educational implementation. It does not replace insurer-specific IFRS 17 systems, accounting policy papers, actuarial methodology approvals, or independently reviewed reporting tools.

3. Product and Portfolio Description

The product modelled is a simplified term-life insurance contract. A term-life contract generally pays a death benefit if the insured life dies during the coverage period.

The model projects policies at policy-year level and then aggregates results for valuation, grouping, scenario, and reporting purposes.

Key product features represented in the engine include sum assured, premiums, mortality, lapse, expenses, reinsurance ceding, reinsurance recovery, counterparty default cost, and annual projection periods.

4. Data, Assumptions, and Synthetic Data Rationale

The project uses synthetic policy data and sample assumptions. This is reasonable for an actuarial science portfolio project because real policy-level life insurance data is usually confidential.

Real insurance datasets may contain sensitive personal, demographic, financial, underwriting, claims, lapse, premium, and company experience information. For this reason, public GitHub actuarial projects commonly rely on synthetic data.

Synthetic data allows the model to demonstrate architecture, methodology, reproducibility, and coding structure without exposing confidential company information. It does not prove that the assumptions are calibrated to a real insurer portfolio.

5. System Architecture

The repository is organised around a modular actuarial pipeline. Configuration and assumption inputs drive projection, curve generation, cashflow calculation, valuation, grouping, scenario analysis, output production, audit metadata, and testing.

This separation of responsibilities improves readability, reviewability, and maintainability. It also reflects the way actuarial production models are often conceptualised, even though this project is educational rather than production-grade.

Main modules

Module	Responsibility
engine/projection.py	Synthetic master data and policy-year projection
engine/curves.py	Discount and lapse curves
engine/cashflows.py	Premiums, claims, expenses, reinsurance, and PV cashflows
engine/bel.py	Best Estimate Liability
engine/ra.py	Risk Adjustment
engine/csm.py	CSM calculation and roll-forward
engine/grouping.py	IFRS 17-style grouping and onerous classification
engine/scenarios.py	Scenario analysis
engine/outputs.py	CSV, Excel, audit, and registry outputs
engine/audit.py	Assumption snapshot, audit report, and run metadata

6. Projection Methodology

The engine creates a policy-year grid. For each policy and projection year, it calculates current age, in-force status, mortality rate, lapse rate, survival multiplier, opening survival ratio, sum assured, issue year, and portfolio identifier.

```
survival_multiplier(t) = (1 - qx(t)) * (1 - lapse_rate(t))
survival_ratio(t) = product of survival_multiplier from year 1 to year t - 1
Therefore, Year 1 survival_ratio = 1
```

This convention is important because year 1 begins with a full opening exposure before mortality and lapse decrements are applied.

7. Cashflow Methodology

The cashflow module calculates expected insurance cashflows at policy-year level. The model includes gross premium inflow, net premium inflow, death benefits, surrender benefit, operating expenses, reinsurance ceding, reinsurance recovery, counterparty default cost, total inflows, total outflows, and net cashflow.

Component	Direction	Description
Gross premium inflow	Inflow	Premium received before deductions
Net premium inflow	Inflow	Premium after relevant adjustments
Death benefit	Outflow	Expected claim payment on death
Surrender benefit	Outflow	Benefit payable on lapse or surrender
Operating expenses	Outflow	Policy administration costs
Reinsurance ceding	Outflow	Cost ceded to reinsurer
Reinsurance recovery	Inflow	Expected recovery from reinsurer
Counterparty default cost	Outflow	Expected cost of reinsurer default

8. Discounting Methodology

The model uses separate timing factors for opening and closing cashflows. This improves actuarial interpretability because premiums are commonly treated as opening cashflows, while claims and expenses may be treated as end-of-period cashflows.

```
discount_factor(t) = 1 / (1 + r)^t  
discount_factor_opening(t) = 1 / (1 + r)^(t - 1)
```

Cashflow type	Timing convention
Premium inflows	Opening discount factor
Reinsurance ceding	Opening discount factor
Claims	Closing discount factor
Expenses	Closing discount factor
Surrender benefits	Closing discount factor
Reinsurance recoveries	Closing discount factor
Counterparty default costs	Closing discount factor

9. Best Estimate Liability Methodology

The engine uses a liability-positive BEL convention. Under this convention, positive BEL represents a net liability and negative BEL represents a net asset.

```
BEL = PV(outflows) - PV(inflows)  
BEL > 0 means net liability  
BEL < 0 means net asset
```

The BEL calculation also includes adjusted mortality:

```
adjusted_qx = qx * (1 - lapse_mortality_correlation * lapse_rate)
```

This feature introduces a simplified relationship between lapse behaviour and mortality exposure. In production, this relationship would require empirical support and governance.

10. Risk Adjustment Methodology

The Risk Adjustment uses a simplified stochastic confidence-level method. It is an educational method and should not be interpreted as a formally approved IFRS 17 production methodology.

```
RA = CoC ratio * max(quantile(PV liability) - mean(PV liability), 0)
```

- Risk drivers include mortality volatility, lapse volatility, expense volatility, and counterparty default events.
- A production RA method would require calibration, confidence-level disclosure, governance approval, validation, and independent review.

11. Contractual Service Margin Methodology

Fulfilment cashflows are represented by BEL plus RA. The initial CSM defers expected unearned profit, while onerous loss is recognised when fulfilment cashflows are positive under the liability-positive convention.

```
total_liability = BEL + RA  
initial_csm = max(-(BEL + RA), 0)  
onerous_loss = max(BEL + RA, 0)
```

BEL already includes premium inflows, so premiums are not subtracted again in the CSM formula. This avoids double-counting premium inflows.

```
coverage_unit(t) = in_force(t) * survival_ratio(t) * (1 - lapse_rate(t))
```

12. IFRS 17 Grouping and Onerous Contract Testing

The grouping module assigns IFRS 17-style groups using portfolio group, annual cohort, profitability bucket, mortality/lapse/reinsurance risk group, and onerous status.

Profitability bucket	Interpretation
Onerous	Contracts expected to produce a loss
No significant possibility of becoming onerous	Profitable contracts with lower onerous risk
Other profitable	Profitable contracts not classified in the previous category

13. Scenario Analysis

The model supports scenario analysis by rebuilding projection, cashflows, BEL, RA, and CSM under shocked assumptions.

Scenario type	Expected interpretation
Mortality increase	Higher expected claims and potentially higher BEL
Lapse increase	Impact depends on profitability and future premium pattern
Expense increase	Higher expected outflows and higher BEL
Discount-rate decrease	Higher present value of future liability cashflows

14. Audit, Governance, and Reproducibility Features

The engine can produce `assumption_snapshot.json`, `audit_report.json`, and `run_registry.csv`. These include run ID, creation timestamp, assumption hash, methodology version, approval status, locked-in discount rate disclosure, row counts, reconciliation totals, and data lineage paths.

These are lightweight governance outputs, not a full production approval workflow. They demonstrate awareness of reproducibility and auditability.

Output	Purpose
<code>assumption_snapshot.json</code>	Stores assumptions used in a model run
<code>audit_report.json</code>	Summarises run metadata and validation information
<code>run_registry.csv</code>	Records run-level registry information

15. Validation and Testing Framework

The automated test suite currently has 17 passing tests. It can be run using:

```
python -m pytest -q
Expected result: 17 passed
```

#	Validation test area
1	module imports
2	discount curve uniqueness
3	projection schema
4	discount curve merge integrity
5	cashflow and BEL smoke checks
6	float64 precision mode

#	Validation test area
7	PV cashflow component timing
8	initial CSM formula
9	grouping policy counts
10	opening survival ratio identity
11	premium and claim discount timing
12	BEL reconciliation to adjusted liability cashflows
13	CSM release allocation
14	mortality shock direction
15	audit snapshot determinism
16	audit registry creation
17	IFRS 17-style grouping fields

16. Key Strengths of the Project

- Clear modular architecture aligned with actuarial modelling components.
- Meaningful IFRS 17 awareness without overstating production readiness.
- Appropriate Python stack for actuarial analytics and model prototyping.
- Automated testing discipline across projection, cashflow, BEL, CSM, grouping, scenarios, and audit outputs.
- Professional documentation and reproducible output structure.

17. Limitations and Future Improvements

Limitation	Possible mitigation
Policy data is synthetic	Disclose clearly and focus on methodology demonstration
Assumptions are not calibrated to insurer experience	Add sample experience study or calibration framework
Mortality and lapse behaviour are simplified	Add duration, age, underwriting, and product-specific assumptions
Risk Adjustment is educational	Add formal confidence-level methodology and validation
CSM methodology is simplified	Add CSM movement, loss component, and experience adjustment logic
Grouping is IFRS 17-style	Add company-specific grouping policy and annual cohort governance
No full disclosure package	Add IFRS 17 disclosure tables and movement analysis
Audit outputs are lightweight	Add sign-off workflow and change logs
No independent peer review	Add model validation checklist and reviewer comments

Future improvements

Priority	Improvement	Rationale
1	Add GitHub Actions for automated tests	Demonstrates professional software workflow
2	Add output reconciliation report	Improves actuarial control evidence
3	Add CSM movement table	Makes IFRS 17 reporting more complete
4	Add assumption calibration examples	Strengthens actuarial credibility

Priority	Improvement	Rationale
5	Add model validation checklist	Improves governance presentation
6	Add sensitivity dashboard	Improves interpretability for reviewers

18. Conclusion

The IFRS 17 Term Life Valuation Engine is a strong actuarial science portfolio project. It demonstrates practical Python modelling ability, simplified IFRS 17 valuation concepts, policy-level projection, actuarial cashflow modelling, BEL, Risk Adjustment, CSM, grouping, scenarios, audit outputs, and automated validation discipline.

The project is most credible when presented honestly as an academic and CV portfolio project rather than a production IFRS 17 reporting system. With further improvements in calibration, validation, CSM movement analysis, and reporting outputs, it could become an even stronger demonstration of actuarial modelling readiness.

Appendix A: Main Configuration Parameters

Parameter	Description
Number of policies	Size of synthetic portfolio
Projection years	Number of future years projected
Locked-in discount rate	Rate used for locked-in measurement concepts
Current discount rate	Rate used for current valuation
Premium amount	Policy premium assumption
Expense amount	Operating expense assumption
Cost-of-capital ratio	Scaling parameter for simplified RA
Mortality assumption	Base mortality rate or table
Lapse assumption	Base lapse rate or curve
Scenario shocks	Mortality, lapse, expense, and discount-rate shocks

Appendix B: Output Files

Output	Purpose
Projection output	Policy-year projection details
Cashflow output	Expected cashflows by policy and year
BEL output	Best Estimate Liability results
RA output	Risk Adjustment results
CSM output	Initial CSM and roll-forward results
Grouping output	IFRS 17-style group classifications
Scenario output	Scenario valuation results
Audit JSON files	Assumption snapshots and audit metadata
Run registry	Registry of model runs

Appendix C: Test Coverage Summary

The test suite supports core mechanical consistency checks. These tests do not prove production readiness, but they provide evidence of modelling discipline and regression control.

Appendix D: CV Project Description

Developed a modular Python-based IFRS 17-style term-life valuation engine covering policy-level projection, mortality/lapse assumptions, cashflows, discounting, BEL, Risk Adjustment, CSM, onerous testing, grouping, scenarios, audit outputs, and pytest validation.